Rooted Induced Trees and the Radius: A Proof of Conjecture 84 of *Written on the Wall II*

Pendant-path unrooting, coherent roots, and sibling propagation

Keio

25 October 2026

Abstract

For a graph G let $\mathrm{tree}(G)$ be the largest number of vertices of an induced subgraph of G that is a tree. Conjecture 84 of DeLaViña's *Written on the Wall II* (a list of conjectures generated by the program Graffiti.pc) asserts that every connected graph G with minimum degree $\delta(G) \geq 1$ satisfies

$$\mathrm{tree}(G) \;\geq\; \frac{2\,\mathrm{rad}(G)}{\delta(G)}.$$

The conjecture is listed as open in *Written on the Wall II* and in Google DeepMind's Formal Conjectures repository. We prove it. For $\delta \geq 2$ the inequality is easy. The real content is the case $\delta = 1$: there it asks for $2\,\mathrm{rad}(G)$ vertices, one more than the classical Erdős–Saks–Sós bound $\mathrm{tree}(G) \geq 2\,\mathrm{rad}(G) - 1$, which is sharp for even cycles.

We prove a stronger statement: *every connected graph with a cut vertex has* $\mathrm{tree}(G) \geq 2\,\mathrm{rad}(G)$. Paths with an even number of vertices show that this is best possible. It follows that every graph attaining the Erdős–Saks–Sós bound with at least three vertices is 2-connected. The bound cannot be achieved with induced paths alone.

The engine is a new *rooted induced-tree lemma*, a tree analogue of the Bacsó–Tuza Induced Path Lemma. Suppose a vertex x is at distance at least t from every vertex of eccentricity at most k, where $1 \leq t \leq k$. Then either G contains an induced tree on $2k+2$ vertices, or x lies in an induced tree on $k+t+1$ vertices whose edges at x all point into a single component of $G - x$.

The proof combines three ideas.

1. *Pendant-path unrooting.* An equivalence identifies the rooted hypothesis with the statement that attaching a pendant path of length $k - t + 1$ at x produces radius at least $k + 1$.
2. *Coherent roots.* These make the Bacsó–Tuza separator exchange work for trees.
3. *Sibling propagation.* In a minimal counterexample every neighbour of the root carries a pendant path of the same length, and so is the root of another minimal counterexample on the same number of vertices. Propagating this through the graph forces a corona structure, in which a comb is already too large.

The whole argument is self-contained. It was checked by computer on all 11 989 763 connected graphs with 2 to 10 vertices (about 1.5×10^8 rooted instances). An independently written checker covered all 1 006 700 565 connected graphs with 11 vertices, and further tests used random graphs. All programs are listed in the appendix.

Contents

1 Introduction **2**

1.1 Induced trees and the radius 2

1.2 Conjecture 84 2

1.3 Main results 3

1.4 The method: a new problem, a new method, an old conjecture 3
1.5 Verification 4

2 Preliminaries **4**

3 Pendant paths: unrooting a rooted hypothesis **5**

4 The rooted induced-tree lemma **6**
4.1 Minimal counterexamples 6
4.2 Unrooting and siblings 7

5 Proofs of the main results **9**

6 Computational verification **9**

7 Open problems **11**

A Source code **12**
A.1 `tlemD.c`: exhaustive check of Theorems 1.6 and 1.3 12
A.2 `crosscheck.py`: independent re-implementation 13
A.3 `sublemmas.py`: randomized checks of the auxiliary steps 13
A.4 `gen_sparse.py`: random sparse connected graphs 15
A.5 `lemd3.c`: independent checker used for $n = 11$ 15
A.6 `gen_sparse2.py`: larger sparse graphs 17
A.7 `tbt.c`: evidence for Conjecture 7.1 17
A.8 `tdiam.c`: evidence for Conjecture 7.2 17

1 Introduction

1.1 Induced trees and the radius

All graphs are finite and simple. For a graph G, an *induced tree* is a vertex set $T \subseteq V(G)$ such that $G[T]$ is a tree, and

$$\operatorname{tree}(G) = \max\{|T| : T \text{ is an induced tree of } G\}.$$

The study of $\operatorname{tree}(G)$ goes back to Erdős, Saks and Sós [4]. Among other results, they showed that a connected graph of radius r contains an induced path on $2r-1$ vertices; in particular

$$\operatorname{tree}(G) \;\geq\; 2\operatorname{rad}(G) - 1 \qquad (G \text{ connected}). \tag{1}$$

The bound is sharp: the cycle C_{2r} has radius r, and its largest induced trees are paths on $2r-1$ vertices. Erdős, Saks and Sós also relate $\operatorname{tree}(G)$ to the connectivity of G. We could not consult the full text of [4] during this work, but we know of no result there or elsewhere that implies Theorem 1.3 below; such a result would already have settled Conjecture 1.1. Bacsó and Tuza [1] proved an *Induced Path Lemma* that refines this. In the form formalised in [8] it reads as follows. If a vertex x is at distance at least t from every vertex of eccentricity at most k, where $1 \leq t \leq k$, then either G has an induced path on $2k+1$ vertices or x is an end of an induced path on $k+t+1$ vertices.

1.2 Conjecture 84

The program Graffiti.pc of DeLaViña [2] produced a large number of conjectures, collected in *Written on the Wall II* (WOWII) [3]. Many of them bound $\operatorname{tree}(G)$ by other invariants. Conjecture 84 reads:

Conjecture 1.1 (WOWII, Conjecture 84)**.** *If G is a simple connected graph, then* $\operatorname{tree}(G) \geq 2\operatorname{rad}(G)/\delta(G)$.

Here $\delta(G)$ is the minimum degree, so the statement concerns connected graphs with at least two vertices. WOWII marks the conjecture as open. The Formal Conjectures repository of Google DeepMind [5] formalises it in Lean (file `GraphConjecture84.lean`) and tags it as an open research problem; we checked this at the repository state of 23 October 2026. Several induced-tree conjectures of WOWII were settled in 2026 [6, 8], but we found no proof of Conjecture 1.1 in the literature. For other bounds on induced trees see, for example, [7].

When $\delta(G) \geq 2$ the conjecture is weak: a diametral geodesic gives $\operatorname{tree}(G) \geq \operatorname{diam}(G) + 1 \geq \operatorname{rad}(G) + 1$. The substance is the case $\delta(G) = 1$, in which the conjecture asks for an induced tree on $2\operatorname{rad}(G)$ vertices. That is one vertex more than (1) guarantees in general. The extremal examples for (1), the even cycles, have no vertex of degree one, and the question is whether a single leaf always buys the extra vertex.

1.3 Main results

Theorem 1.2. *Conjecture 1.1 is true. Every connected graph G with $\delta(G) \geq 1$ satisfies* $\operatorname{tree}(G) \geq 2\operatorname{rad}(G)/\delta(G)$. *For $\delta(G) = 1$ the bound is attained by every path on an even number of vertices.*

Theorem 1.2 follows from a structural strengthening of (1).

Theorem 1.3 (Cut-vertex theorem)**.** *If G is connected and has a cut vertex, then* $\operatorname{tree}(G) \geq 2\operatorname{rad}(G)$.

Corollary 1.4. *If G is connected, has at least three vertices, and attains (1) with equality, then G is 2-connected.*

Remark 1.5. Theorem 1.3 does not hold for induced paths. Let $C_4 \circ K_1$ be the 4-cycle with one pendant vertex attached to each of its vertices. It has radius 3 and has cut vertices, but its longest induced path has 5 vertices. Its largest induced trees, for instance three consecutive cycle vertices together with their three pendant vertices, have $6 = 2\operatorname{rad}$ vertices. Theorem 1.3 therefore genuinely needs trees.

To state the central tool, write $\mathcal{C}_k(G) = \{v : \operatorname{ecc}_G(v) \leq k\}$ for the *k-centre* of a connected graph G. Given integers $k \geq 1$ and $t \geq 1$, a vertex x is *(k,t)-far* in G if $d_G(x,z) \geq t$ for every $z \in \mathcal{C}_k(G)$; this holds vacuously when $\mathcal{C}_k(G) = \emptyset$. An induced tree T is *x-coherent* if $x \in T$ and all neighbours of x in T lie in one connected component of $G - x$.

Theorem 1.6 (Rooted induced-tree lemma)**.** *Let G be connected, let $1 \leq t \leq k$ be integers, and let x be (k,t)-far in G. Then at least one of the following holds:*

(T) *G has an induced tree on at least $2k+2$ vertices;*

(R) *G has an x-coherent induced tree on at least $k+t+1$ vertices.*

Compared with the Bacsó–Tuza lemma, the unrooted alternative is raised from $2k+1$ to $2k+2$ vertices. In exchange, the rooted alternative is a coherent tree instead of a path ending at x. Both bounds in Theorem 1.6 are sharp (Remark 4.1). Whether the rooted alternative can always be taken to be an induced path ending at x is open; see Conjecture 7.1, which we verified for all graphs with at most 10 vertices.

1.4 The method: a new problem, a new method, an old conjecture

We follow the scheme *new problem → new method → application*. The new problem is Theorem 1.6. The application is Conjecture 1.1, through Theorem 1.3. The method has three parts.

1. Pendant-path unrooting (an equivalence). Attach to x a new pendant path of length $m = k - t + 1$ and call the result F. Then x is (k, t)-far in G if and only if $\mathrm{rad}(F) \geq k + 1$ (Lemma 3.2). If moreover x is not a cut vertex of G, then the conclusion of Theorem 1.6 for (G, x) is equivalent to $\mathrm{tree}(F) \geq 2k + 2$ (Lemma 4.5). So a *rooted* statement about (G, x) becomes an *unrooted* statement about F, in which the root is just one of possibly many vertices that carry a pendant path of length m. A leaf is a pendant path of length 1, and in this language the leaf case of Conjecture 1.1 is precisely the case $t = k$ of Theorem 1.6.

2. Coherent roots. The Bacsó–Tuza induction removes a neighbour s of the root. When s is a cut vertex, it glues a rooted object found at s to an object on the other side of s. For paths this gluing is automatic. For trees it fails as soon as a tree branches into both sides of s. Coherence, meaning that all tree edges at the root point into one component, is exactly the property that makes the gluing work. It also allows a new reduction when the root itself is a cut vertex (Lemma 4.3).

3. Sibling propagation. After the reductions, a minimal counterexample is rigid. Every neighbour s of the root x is a cut vertex whose far side is a pendant path of length exactly m (Lemma 4.4). In the unrooted graph F the root and its neighbours therefore look alike: all of them carry pendant paths of length m. By the two equivalences, each of them is the root of another counterexample, with the same number of vertices and the same parameters, which we call a *sibling* (Lemma 4.6). Each sibling is again minimal, so its own neighbours carry pendant paths of length m too. The structure spreads through the whole graph: F is a connected graph K with a pendant path of length m at every vertex. In such a graph a diametral geodesic of K together with its pendant paths is an induced tree on at least $(t+1)(m+1) = 2k+2+(t-1)(m-1)$ vertices, which contradicts minimality.

The second half of the argument uses induction on the number of vertices in a non-obvious way. The sibling instances are *not* smaller than the original one. They are counterexamples of the same size, so the structure theorem for minimal counterexamples applies to them as well.

1.5 Verification

Besides the proofs, we checked Theorem 1.6 by computer for every connected graph with 2 to 10 vertices, every root x and every k, using the strongest admissible t. That is 11 989 763 graphs and 153 437 850 rooted instances. We checked Theorem 1.3 on all 2 044 493 such graphs with a cut vertex. A second, independently written program reproduces the counts for up to 8 vertices. A third, also independent, checker confirmed Theorems 1.6 and 1.3 for all 1 006 700 565 connected graphs with 11 vertices. Random graphs with up to 16 vertices and randomized tests of every auxiliary step of the proof gave no exception. Section 6 has the details and the appendix lists the code.

2 Preliminaries

For a connected graph G and $u, v \in V(G)$ we write $d(u, v) = d_G(u, v)$, $\mathrm{ecc}_G(v) = \max_w d(v, w)$, $\mathrm{rad}(G) = \min_v \mathrm{ecc}_G(v)$ and $\mathrm{diam}(G) = \max_v \mathrm{ecc}_G(v)$. For $S \subseteq V(G)$, $G[S]$ is the induced subgraph and $G - S = G[V(G) \setminus S]$. For an induced tree T and $x \in T$, $N_T(x) = N_G(x) \cap T$. A *geodesic* is a shortest path. A vertex c is a *cut vertex* of a connected graph G if $G - c$ is disconnected.

Lemma 2.1 (Transfer)**.** *Let G' be a connected induced subgraph of G with $x \in V(G')$. Every induced tree of G' is an induced tree of G, and every x-coherent induced tree of G' is x-coherent in G.*

Proof. $G[T] = G'[T]$ for $T \subseteq V(G')$. Each component of $G' - x$ is connected in $G - x$, so it lies inside one component of $G - x$. □

Two trivial cases of coherence are worth recording. If x is a leaf of $G[T]$, then T is x-coherent. If $G - x$ is connected, then every induced tree containing x is x-coherent.

Lemma 2.2 (Separators)**.** *Let s be a cut vertex of a connected graph G, let A be the union of some components of $G - s$, and let $B = V(G) \setminus (A \cup \{s\})$.*

(a) *No edge joins A to B. Distances between vertices of $A \cup \{s\}$ are the same in G and in $G[A \cup \{s\}]$, and $d(a, b) = d(a, s) + d(s, b)$ for $a \in A \cup \{s\}$, $b \in B \cup \{s\}$.*

(b) *If $T_1 \subseteq A \cup \{s\}$ and $T_2 \subseteq B \cup \{s\}$ are induced trees containing s, then $T_1 \cup T_2$ is an induced tree with $|T_1| + |T_2| - 1$ vertices.*

(c) *If T is an induced tree containing s and $N_T(s)$ lies in one component D of $G - s$, then $T \subseteq D \cup \{s\}$.*

(d) *Every geodesic from s to a vertex $b \in B$ has all its vertices other than s in the component of $G - s$ that contains b.*

Proof. (a) Components of $G - s$ are not joined by edges. A path between two vertices of $A \cup \{s\}$ that leaves $A \cup \{s\}$ must pass through s twice, so it is not a shortest path. Every path from A to B passes through s. (b) $G[T_1 \cup T_2]$ is the union of the trees $G[T_1]$ and $G[T_2]$, which share only s. By (a) there are no further edges. (c) Every component of $G[T] - s$ contains a vertex of $N_T(s)$ and is connected in $G - s$. (d) The geodesic minus s is connected in $G - s$. □

Lemma 2.3. *Let s be a cut vertex of a connected graph G, let $B \neq \emptyset$ be a union of components of $G - s$, and let $u \in B$ maximise $d(s, u)$ over B. Then $G - u$ is connected.*

Proof. Let $v \neq u$ and let P be a geodesic from s to v. If $u \notin P$, then v is joined to s in $G - u$. Otherwise u lies strictly before v on P. By Lemma 2.2(d), v lies in the component of u, so $v \in B$ and $d(s, v) > d(s, u)$, contradicting the choice of u. □

Geodesics are induced paths. We use this freely, together with the following consequence of Lemma 2.2(b): if s is a cut vertex and e_1, e_2 are the depths $\max d(s, \cdot)$ of two different components of $G - s$, then G has an induced path on $e_1 + e_2 + 1$ vertices through s.

3 Pendant paths: unrooting a rooted hypothesis

Definition 3.1. Let F be connected, $b \in V(F)$ and $m \geq 1$. A set $W \subseteq V(F) \setminus \{b\}$ is a *pendant path of length m at b* if W is a connected component of $F - b$, $|W| = m$, and $F[W \cup \{b\}]$ is a path with end vertex b. For a graph H and $v \in V(H)$, $H \oplus_v P_m$ denotes H with m new vertices $p_1, \ldots, p_m$ and new edges $vp_1, p_1p_2, \ldots, p_{m-1}p_m$. If W is a pendant path of length m at b in F, then $F \cong (F - W) \oplus_b P_m$.

Lemma 3.2 (Pendant-path equivalence)**.** *Let H be connected, $v \in V(H)$, let $1 \leq t \leq k$ be integers, $m = k - t + 1$, and $F = H \oplus_v P_m$. Then*

$$\operatorname{rad}(F) \geq k + 1 \iff v \text{ is } (k, t)\text{-far in } H.$$

Proof. For $z \in V(H)$ we have $d_F(z, w) = d_H(z, w)$ for $w \in V(H)$ and $d_F(z, p_h) = d_H(z, v) + h$. Hence

$$\operatorname{ecc}_F(z) = \max\{\operatorname{ecc}_H(z),\ d_H(z, v) + m\} \quad (z \in V(H)), \qquad \operatorname{ecc}_F(p_h) = \max\{m - h,\ h + \operatorname{ecc}_H(v)\}.$$

($\Rightarrow$) Let $z \in \mathcal{C}_k(H)$. Then $\operatorname{ecc}_F(z) \geq k + 1 > \operatorname{ecc}_H(z)$, so $d_H(z, v) + m \geq k + 1$, i.e. $d_H(z, v) \geq t$.

($\Leftarrow$) Since $d(v, v) = 0 < t$, $v \notin \mathcal{C}_k(H)$, so $\operatorname{ecc}_H(v) \geq k + 1$ and $\operatorname{ecc}_F(p_h) \geq k + 2$. If $z \in V(H)$ has $\operatorname{ecc}_H(z) \leq k$, then $d_H(z, v) \geq t$ and $\operatorname{ecc}_F(z) \geq t + m = k + 1$. □

Remark 3.3. If u is a leaf of G with neighbour w, then $G = (G-u) \oplus_w P_1$. For $r \geq 2$, Lemma 3.2 with $k = t = r-1$ shows that $\mathrm{rad}(G) \geq r$ if and only if w is $(r-1, r-1)$-far in $G-u$. In this sense the leaf case of Conjecture 1.1 is the diagonal case $t = k$ of Theorem 1.6.

4 The rooted induced-tree lemma

Remark 4.1 (Sharpness)**.** Take $G = C_{2k+2}$. Then $\mathrm{rad}(G) = k+1$, $\mathcal{C}_k(G) = \emptyset$, and every vertex is (k,k)-far. Here (T) fails, because $\mathrm{tree}(C_{2k+2}) = 2k+1$, and (R) holds with exactly $k+t+1 = 2k+1$ vertices. Now take the path on $2k+1$ vertices with x an end vertex. Then $\mathcal{C}_k = \{\text{centre}\}$, x is (k,k)-far, (T) fails, and (R) holds with equality. So neither $2k+2$ nor $k+t+1$ can be increased.

4.1 Minimal counterexamples

Suppose Theorem 1.6 is false. A *counterexample* is a quadruple (G, x, k, t) satisfying the hypotheses of the theorem for which both (T) and (R) fail. Choose a counterexample with $|V(G)|$ minimum and, subject to this, t minimum; k is arbitrary. Every counterexample with the same values of $|V(G)|$ and t is then also minimal. By minimality:

(∗) every instance (G', x', k', t') of Theorem 1.6 with $|V(G')| < |V(G)|$, or with $|V(G')| = |V(G)|$ and $t' < t$, satisfies (T) or (R).

In particular, by Lemma 2.1: *if G' is a connected induced subgraph of G with fewer vertices, $x \in V(G')$, and x is (k,t)-far in G', then (G, x, k, t) satisfies (T) or (R).* We call this a *reduction to G'*. Throughout this section (G, x, k, t) is a minimal counterexample and $m = k - t + 1$.

Lemma 4.2. *$t \geq 2$. Consequently $1 \leq m \leq k-1$, $\mathrm{ecc}_G(x) \geq k+1$, and x has a neighbour.*

Proof. Since $d(x,x) = 0 < t$, $x \notin \mathcal{C}_k(G)$, so $\mathrm{ecc}_G(x) \geq k+1$. If $t = 1$, a geodesic from x to a vertex at distance $k+1$ is an induced path on $k+2 = k+t+1$ vertices with end x. It is x-coherent, so (R) holds. Hence $t \geq 2$. □

Lemma 4.3 (The root is not a cut vertex)**.** *$G - x$ is connected.*

Proof. Suppose not. Let K_1 be a component of $G-x$ of maximum depth $e_1 = \max_{v \in K_1} d(x,v) = \mathrm{ecc}(x) \geq k+1$. Let $e' \geq 1$ be the largest depth of the remaining components, attained by a component K_2. Joining geodesics from x into K_1 and into K_2 gives an induced path on $e_1 + e' + 1$ vertices (Lemma 2.2(b)). Since (T) fails, $e' \leq 2k - e_1 \leq k-1$.

Let $G_1 = G[K_1 \cup \{x\}]$ and $\tau = \min\{t,\ k+1-e'\}$, so that $1 \leq \tau \leq k$. By Lemma 2.2(a), for $z \in K_1 \cup \{x\}$,

$$\mathrm{ecc}_G(z) = \max\{\mathrm{ecc}_{G_1}(z),\ d(z,x) + e'\},$$

and distances inside $K_1 \cup \{x\}$ agree in G and G_1. If some $z \in \mathcal{C}_k(G_1)$ had $d(x,z) \leq \tau - 1$, then $\mathrm{ecc}_G(z) \leq \max\{k, \tau - 1 + e'\} \leq k$. So $z \in \mathcal{C}_k(G)$ with $d(x,z) < t$, which is impossible. Hence x is (k,τ)-far in G_1.

As $|V(G_1)| < |V(G)|$, (∗) applies to (G_1, x, k, τ). If (T) holds there, it holds in G. Otherwise there is an x-coherent induced tree T of G_1 with $|T| \geq k + \tau + 1$.

- If $\tau = t$, then T satisfies (R) in G, because its neighbours of x lie in K_1.
- If $\tau = k+1-e'$, then T together with a geodesic of length e' from x into K_2 is an induced tree (Lemma 2.2(b)) on at least $k + \tau + 1 + e' = 2k+2$ vertices.

Either way we contradict the choice of (G, x, k, t). □

Lemma 4.4 (Neighbours of the root)**.** *Let s be a neighbour of x. Then s is a cut vertex of G, and $G - s$ has exactly two components: $X_s \ni x$, and a pendant path U_s of length m at s.*

Proof. (a) s is a cut vertex. Suppose $G - s$ is connected. We claim that x is (k,t)-far in $G - s$. Let $y \in \mathcal{C}_k(G - s)$ with $d_{G-s}(x,y) < t$. Then $d_G(y,s) \le d_G(y,x) + 1 \le t \le k$, and all other distances from y are at most k, because they are at most the corresponding distances in $G - s$. Hence $y \in \mathcal{C}_k(G)$ and $d_G(x,y) < t$, which is impossible. This proves the claim, and a reduction to $G - s$ gives a contradiction.

So s is a cut vertex. Let X be the component of $G - s$ containing x, and let $U = V(G) \setminus (X \cup \{s\}) \neq \emptyset$.

(b) A coherent tree at s. For $z \in \mathcal{C}_k(G)$ we have $d(s,z) \ge d(x,z) - 1 \ge t - 1 \ge 1$, so s is $(k, t-1)$-far. By $(*)$ (same graph, smaller t), $(G, s, k, t-1)$ satisfies (T) or (R). Since (T) fails for G, there is an s-coherent induced tree T_s with $|T_s| \ge k + t$. By Lemma 2.2(c), $T_s \subseteq D \cup \{s\}$ for one component D of $G - s$. If $D \subseteq U$, then x is adjacent to s and to no vertex of D, so $T_s \cup \{x\}$ is an induced tree with x as a leaf on at least $k + t + 1$ vertices: (R) holds, which is impossible. Hence $T_s \subseteq X \cup \{s\}$.

(c) Depth of U at most m. Let $u \in U$ maximise $\mu = d(s,u)$ over U, and let $q = (s = q_0, q_1, \ldots, q_\mu = u)$ be a geodesic, so $q_1, \ldots, q_\mu \in U$ by Lemma 2.2(d). By Lemma 2.2(b), $T_s \cup V(q)$ is an induced tree on $|T_s| + \mu$ vertices. Since (T) fails, $|T_s| + \mu \le 2k + 1$, so $\mu \le m$.

(d) Depth of U at least m. By Lemma 2.3, $G - u$ is connected. If x were (k,t)-far in $G - u$, a reduction to $G - u$ would give a contradiction. So there is $y \in \mathcal{C}_k(G - u)$ with $d_{G-u}(x,y) \le t - 1$. If $d_G(y,u) \le k$, then $y \in \mathcal{C}_k(G)$ and $d_G(x,y) \le t - 1$, which is impossible. Thus

$$k + 1 \le d_G(y,u) \le d(y,x) + d(x,s) + d(s,u) \le (t-1) + 1 + \mu,$$

so $\mu \ge m$. Hence $\mu = m$.

(e) U is a pendant path. Since $d(x,s) = 1 < t$, $s \notin \mathcal{C}_k(G)$, so $\mathrm{ecc}_G(s) \ge k + 1$. By Lemma 2.2(a), $\mathrm{ecc}_G(s) = \max\{e_X, \mu\}$, where $e_X = \max_{v \in X} d(s,v)$. As $\mu = m \le k - 1$, we get $e_X \ge k + 1$.

Let $\tilde{G} = G[X \cup \{s\} \cup V(q)]$, a connected induced subgraph containing x. Lemma 2.2(a), applied in G and in $\tilde{G}$, gives $\mathrm{ecc}_{\tilde{G}}(z) = \mathrm{ecc}_G(z)$ for $z \in X \cup \{s\}$: both equal $\max\{\max_{v \in X \cup \{s\}} d(z,v),\ d(z,s) + m\}$. For $h \ge 1$ we have $\mathrm{ecc}_{\tilde{G}}(q_h) \ge h + e_X \ge k + 2$. So $\mathcal{C}_k(\tilde{G}) = \mathcal{C}_k(G) \cap (X \cup \{s\})$, with the same distances to x, and x is (k,t)-far in $\tilde{G}$. If $\tilde{G} \neq G$, a reduction to $\tilde{G}$ gives a contradiction. Therefore $U = \{q_1, \ldots, q_m\}$. Since q is a geodesic, $G[U \cup \{s\}]$ is the path $sq_1 \cdots q_m$, so U is a single component of $G - s$ and a pendant path of length m at s. □

4.2 Unrooting and siblings

Let $F = G \oplus_x P_m$ with new vertices $W_x = \{p_1, \ldots, p_m\}$; thus W_x is a pendant path of length m at x in F.

Lemma 4.5 (Unrooting)**.** *Let (H, v, k, t) be any instance of Theorem 1.6 in which v is not a cut vertex of H, and let $F_H = H \oplus_v P_{k-t+1}$. Then (H, v, k, t) satisfies (T) or (R) if and only if $\mathrm{tree}(F_H) \ge 2k + 2$. In particular, for our minimal counterexample, $\mathrm{rad}(F) \ge k + 1$ and $\mathrm{tree}(F) \le 2k + 1$.*

Proof. ($\Rightarrow$) Under (T), the tree lies in F_H. Under (R), the coherent tree together with the pendant path is an induced tree on at least $k + t + 1 + (k - t + 1) = 2k + 2$ vertices.

($\Leftarrow$) Let T be an induced tree of F_H with $|T| \ge 2k + 2$. If $T \subseteq V(H)$, (T) holds. Otherwise T contains a pendant vertex. Since T is connected and has more than $k - t + 1$ vertices, it contains v. Moreover T meets the pendant path in an initial segment $p_1, \ldots, p_j$, so $T \cap V(H)$ is an induced tree of H containing v with at least $2k + 2 - (k - t + 1) = k + t + 1$ vertices. It is v-coherent because $H - v$ is connected, so (R) holds.

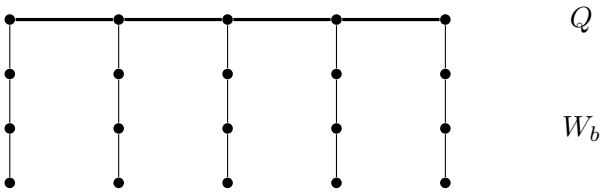


Figure 1: The comb used at the end of the proof: a geodesic Q of the core K with the pendant paths W_b ($b \in Q$) attached. Here $|Q| = 5$ and $m = 3$.

For our counterexample, $\mathrm{rad}(F) \geq k+1$ follows from Lemma 3.2, and $\mathrm{tree}(F) \leq 2k+1$ follows from Lemma 4.3 and the equivalence. □

Lemma 4.6 (Siblings)**.** *Let W be a pendant path of length m at a vertex b of F. Then $(F - W, b, k, t)$ is a minimal counterexample.*

Proof. $F - W$ is connected and $F \cong (F - W) \oplus_b P_m$. Since $\mathrm{rad}(F) \geq k+1$, Lemma 3.2 shows that b is (k,t)-far in $F - W$. If (T) held for $(F - W, b, k, t)$, then $\mathrm{tree}(F) \geq 2k+2$. If (R) held, the coherent tree together with W would be an induced tree of F on at least $2k+2$ vertices. Both contradict Lemma 4.5, so $(F - W, b, k, t)$ is a counterexample. It has $|V(F)| - m = |V(G)|$ vertices and the same t, so it is minimal. □

Consequently Lemmas 4.3 and 4.4 apply to every sibling $(F - W, b, k, t)$. Let B be the set of vertices of F at which F has a pendant path of length m; note $x \in B$. We use repeatedly that a path on N vertices has radius $\lfloor N/2 \rfloor$, while $\mathrm{rad}(F) \geq k+1 \geq m+2$.

Lemma 4.7 (Propagation)**.**

(i) *If W is a pendant path of length m at b, then $W \cap B = \emptyset$.*

(ii) *Every $b \in B$ has exactly one pendant path of length m; call it W_b.*

(iii) *$W_b \cap W_{b'} = \emptyset$ for distinct $b, b' \in B$.*

(iv) *If $b \in B$ and $v \in N_F(b) \setminus W_b$, then $v \in B$.*

Proof. (i) Write $W = \{w_1, \ldots, w_m\}$ with $bw_1 \cdots w_m$ a path, and suppose $w_h \in B$ with pendant path W'. The components of $F - w_h$ are $\{w_{h+1}, \ldots, w_m\}$, which has $m - h < m$ vertices, and $R = V(F) \setminus \{w_h, \ldots, w_m\}$. Hence $W' = R$, $|R| = m$, and $F[R \cup \{w_h\}]$ is a path ending at w_h. Then F is a path on $2m - h + 1 \leq 2m$ vertices, so $\mathrm{rad}(F) \leq m$, a contradiction.

(ii) Let $W \neq W'$ be pendant paths of length m at b. They are distinct components of $F - b$. By Lemma 4.6 and Lemma 4.3, b is not a cut vertex of $F - W$. But W' is a component of $(F - W) - b$, so $V(F) = W \cup \{b\} \cup W'$. Then F is a path on $2m+1$ vertices of radius m, a contradiction.

(iii) By (i), $b' \notin W_b$. The connected set W_b therefore lies in one component of $F - b'$. If it meets $W_{b'}$, then $W_b \subseteq W_{b'}$, and symmetrically $W_{b'} \subseteq W_b$. But the only vertex outside W_b adjacent to W_b is b, and the only vertex outside $W_{b'}$ adjacent to $W_{b'}$ is b'. So $b = b'$.

(iv) By Lemma 4.6, $(F - W_b, b, k, t)$ is a minimal counterexample. By Lemma 4.4 applied to it, $(F - W_b) - v$ has exactly two components: $X_v \ni b$, and a pendant path U_v of length m at v. Since W_b is attached to $F - W_b$ only at $b \in X_v$, the components of $F - v$ are U_v and $X_v \cup W_b$. Thus U_v is a pendant path of length m at v in F, and $v \in B$. □

Proof of Theorem 1.6. Let $K = F - \bigcup_{b \in B} W_b$. By Lemma 4.7(i) and (iii), the sets W_b are pairwise disjoint and disjoint from B, so $B \subseteq V(K)$. K is connected: a geodesic of F between two vertices of K cannot enter any W_b, since it would have to leave through b again. For $b \in B$

and $b' \neq b$, the vertex b has no neighbour in $W_{b'}$: the only vertex outside $W_{b'}$ adjacent to $W_{b'}$ is b', and $b \notin W_{b'}$ by Lemma 4.7(i). Hence $N_K(b) = N_F(b) \setminus W_b \subseteq B$ by Lemma 4.7(iv). Since $x \in B$ and K is connected, $B = V(K)$. Thus F consists of the connected graph K together with a pendant path W_b of length m at every vertex b of K.

Let c be a centre of K. Every vertex of K is within $\mathrm{rad}(K)$ of c, and every vertex of W_b is within $d_K(c,b) + m \leq \mathrm{rad}(K) + m$. So $\mathrm{rad}(F) \leq \mathrm{rad}(K) + m$, and $\mathrm{diam}(K) \geq \mathrm{rad}(K) \geq k + 1 - m = t$.

Let Q be a geodesic of K with $\mathrm{diam}(K) + 1 \geq t + 1$ vertices. Then $Q \cup \bigcup_{b \in Q} W_b$ is an induced tree of F (Figure 1). Indeed, Q is an induced path of $K = F[V(K)]$, and each W_b is attached only at b. Using $k + 1 = t + m$, its order is

$$|Q|(m+1) \;\geq\; (t+1)(m+1) \;=\; 2(t+m) + (t-1)(m-1) \;\geq\; 2k+2.$$

This contradicts $\mathrm{tree}(F) \leq 2k + 1$ (Lemma 4.5). Hence no counterexample exists. □

5 Proofs of the main results

Proof of Theorem 1.3. Let c be a cut vertex and $r = \mathrm{rad}(G)$. List the components of $G - c$ by decreasing depth $e_1 \geq e_2 \geq \cdots$, where the depth of a component A is $\max_{a \in A} d(c,a) \geq 1$. Then $e_1 = \mathrm{ecc}(c) \geq r$. Geodesics from c into the two deepest components form an induced path on $e_1 + e_2 + 1$ vertices. So we may assume $e_1 + e_2 + 1 \leq 2r - 1$, which gives $m := e_2 \leq r - 2$ and hence $r \geq 3$.

Let A_1, A_2 be the two deepest components, let $H = G[A_1 \cup \{c\}]$, and let q be a geodesic of length m from c into A_2. Put $F = G[V(H) \cup V(q)] \cong H \oplus_c P_m$. By Lemma 2.2(a), for $z \in A_1 \cup \{c\}$,

$$\mathrm{ecc}_F(z) = \max\{\mathrm{ecc}_H(z), d(z,c) + e_2\} = \mathrm{ecc}_G(z) \geq r,$$

while the vertices of q other than c have eccentricity at least $e_1 + 1 \geq r + 1$ in F. Hence $\mathrm{rad}(F) \geq r$.

Set $k = r - 1$ and $t = r - m$, so that $2 \leq t \leq k$ and $m = k - t + 1$. By Lemma 3.2, c is (k,t)-far in H, and Theorem 1.6 applies to (H, c, k, t). If (T) holds, then $\mathrm{tree}(G) \geq \mathrm{tree}(H) \geq 2k + 2 = 2r$. If (R) holds, an induced tree $T \ni c$ of H with $|T| \geq k + t + 1 = 2r - m$, together with $V(q)$, is an induced tree of G on at least $2r$ vertices (Lemma 2.2(b)). □

Proof of Theorem 1.2. Let G be connected with $\delta(G) \geq 1$; so $|V(G)| \geq 2$ and $\mathrm{rad}(G) \geq 1$.

If $\delta(G) \geq 2$, a diametral geodesic is an induced path on $\mathrm{diam}(G) + 1$ vertices. Hence $\mathrm{tree}(G) \geq \mathrm{rad}(G) + 1 > \mathrm{rad}(G) \geq 2\,\mathrm{rad}(G)/\delta(G)$.

Let $\delta(G) = 1$. If $G = K_2$, then $\mathrm{tree}(G) = 2 = 2\,\mathrm{rad}(G)$. Otherwise the neighbour of a vertex of degree one is a cut vertex, and Theorem 1.3 gives $\mathrm{tree}(G) \geq 2\,\mathrm{rad}(G) = 2\,\mathrm{rad}(G)/\delta(G)$.

Finally, the path on $2r$ vertices has $\delta = 1$, radius r and tree $= 2r$, so equality holds. □

Proof of Corollary 1.4. A connected graph with at least three vertices that is not 2-connected has a cut vertex. By Theorem 1.3 it then has $\mathrm{tree}(G) \geq 2\,\mathrm{rad}(G) > 2\,\mathrm{rad}(G) - 1$. □

Remark 5.1. In the leaf case one can also go directly through Remark 3.3. If u is a leaf with neighbour w and $r = \mathrm{rad}(G) \geq 2$, then Theorem 1.6 with $k = t = r - 1$, applied to $(G - u, w)$, gives either an induced tree on $2r$ vertices or a w-coherent tree on $2r - 1$ vertices. In the second case adding u gives a tree on $2r$ vertices.

6 Computational verification

The proofs above are complete and do not depend on computation. We nevertheless tested every statement, and every auxiliary step, by computer. All programs are listed in Appendix A. Graphs were generated with `geng` from `nauty` [9].

Exhaustive check of Theorems 1.6 and 1.3. The program `tlemD.c` reads graphs in graph6 format. For each graph it computes all distances and enumerates all vertex subsets with bit masks to find every induced tree. From these it obtains $\mathrm{tree}(G)$ and, for every vertex x, the largest order of an x-coherent induced tree. For every $k \geq 1$ and every x with $t = \min\{k, d(x, \mathcal{C}_k(G))\} \geq 1$ (the strongest admissible t), it checks (T) or (R). It also checks $\mathrm{tree}(G) \geq 2\,\mathrm{rad}(G)$ for every graph with a cut vertex and for every graph with a leaf. Table 1 summarises the results; there were no failures.

n	connected graphs	rooted instances	graphs with a cut vertex	graphs with a leaf
2	1	0	0	1
3	2	2	1	1
4	6	18	3	3
5	21	104	11	10
6	112	811	56	51
7	853	7 800	385	346
8	11 117	119 181	3 994	3 675
9	261 080	3 102 468	67 014	63 308
10	11 716 571	150 207 466	1 973 029	1 908 362
total	11 989 763	153 437 850	2 044 493	1 975 757

Table 1: Exhaustive verification. Every rooted instance satisfies (T) or (R), and every graph with a cut vertex satisfies $\mathrm{tree}(G) \geq 2\,\mathrm{rad}(G)$.

Independent re-implementation. `crosscheck.py` recomputes the same quantities with `networkx`, sharing no code with `tlemD.c`. For $3 \leq n \leq 8$ it reports exactly the instance counts of Table 1 (2, 18, 104, 811, 7 800, 119 181) and no failures.

Eleven vertices. A third checker, `lemd3.c`, was written independently of the other two during an audit of the proof. It enumerates induced trees by a leaf-deletion recursion over vertex subsets. For every rooted instance and every admissible t (not only the largest), it tests (T) or (R), and it tests Theorem 1.3. It skips only instances that are settled outright:

- those where a diametral path gives (T);
- those where a geodesic from x gives (R);
- graphs of diameter at most 2. In such a graph only $k = t = 1$ is admissible, and that case is settled by a geodesic from x. A cut vertex then forces radius 1.

Run on all 1 006 700 565 connected graphs with 11 vertices (split as `geng -cq 11 r/4`, $r = 0, \ldots, 3$), it tested 693 086 400 non-trivial rooted instances and all 105 442 877 graphs that have a cut vertex and diameter at least 3. There was no failure. (Graphs with a cut vertex and diameter at most 2 have a dominating vertex, so Theorem 1.3 is trivial for them.) We re-ran `lemd3.c` ourselves on $n = 8, 9$; again there was no failure. With the diameter-2 shortcut switched off (argument `2`), it reports for $n = 9$ the same number 67 014 of graphs with a cut vertex as Table 1.

Random graphs. We fed `tlemD.c` with 12 000 random graphs $G(15, p)$ with p equal to 0.12, 0.16, 0.20 or 0.25, generated by `genrang` (disconnected ones were skipped), and with 6 000 sparse graphs on 10 to 16 vertices produced by `gen_sparse.py` (random trees plus a few random edges). In total 10 687 connected graphs and 486 271 rooted instances were checked, with no failures. A further 600 sparse connected graphs with 17 to 20 vertices (`gen_sparse2.py`, checked by `tlemDr`

compiled with `MAXN` raised to 22) gave 55 205 rooted instances and 598 graphs with a cut vertex, again with no failure.

Auxiliary steps. `sublemmas.py` tested, on 8 000 random connected graphs with 4 to 22 vertices, the following claims: the equivalence of Lemma 3.2 (164 902 instances); the preservation of farness in Lemma 4.4(a) (52 158); farness in Lemma 4.3 (3 001); connectivity in Lemma 2.3 (6 081); the inequality $\mu \geq m$ of Lemma 4.4(d) (2 729); and farness in $\tilde{G}$ in Lemma 4.4(e) (3 803). No violation occurred.

7 Open problems

Conjecture 7.1 (Path version)**.** *Under the hypotheses of Theorem 1.6, either* $\mathrm{tree}(G) \geq 2k+2$*, or* x *is an end of an induced path on at least* $k+t+1$ *vertices.*

Conjecture 7.1 would imply both Theorem 1.6 and the Bacsó–Tuza lemma. The program `tbt.c` confirms it for all connected graphs with at most 10 vertices: 156 575 328 instances (G, x, k, t) over all admissible t, with no failure. Our proof does not adapt directly, because the unrooting equivalence (Lemma 4.5) fails for paths.

Conjecture 7.2. *If* G *is connected and* $\mathrm{diam}(G) > \mathrm{rad}(G)$*, then* $\mathrm{tree}(G) \geq 2\,\mathrm{rad}(G)$*.*

Every graph with a leaf satisfies diam $>$ rad, so Conjecture 7.2 would contain the leaf case of Theorem 1.3. The program `tdiam.c` confirms it for all 8 052 809 connected graphs with at most 10 vertices and diam $>$ rad. The analogous statement for induced paths fails: the graph $C_4 \circ K_1$ of Remark 1.5 (graph6 code `G?‘FE_`) is a counterexample. Corollary 1.4 and Conjecture 7.2 suggest the following question. *Which 2-connected self-centred graphs attain the Erdős–Saks–Sós bound* $\mathrm{tree}(G) = 2\,\mathrm{rad}(G) - 1$*?* Even cycles and the cocktail-party graphs $K_{2,2,\ldots,2}$ (with at least two parts) do.

Acknowledgement of methods. The reductions in Lemma 4.4(a)–(d) follow the structure of the Bacsó–Tuza induction [1], as formalised in [8]. The coherent-root formulation, the cut-root reduction (Lemma 4.3), the unrooting equivalence (Lemma 4.5) and the sibling propagation (Lemmas 4.6 and 4.7) are new as far as we know.

References

[1] G. Bacsó and Zs. Tuza, A characterization of graphs without long induced paths, *J. Graph Theory* 14 (1990), 455–464.

[2] E. DeLaViña, Graffiti.pc: a variant of Graffiti, in: *Graphs and Discovery*, DIMACS Ser. Discrete Math. Theoret. Comput. Sci. 69, Amer. Math. Soc., 2005, 71–79.

[3] E. DeLaViña, *Written on the Wall II: Conjectures of Graffiti.pc*, `http://cms.uhd.edu/faculty/delavinae/research/wowII/all.html` (Conjecture 84).

[4] P. Erdős, M. Saks and V. T. Sós, Maximum induced trees in graphs, *J. Combin. Theory Ser. B* 41 (1986), 61–79.

[5] Google DeepMind, *Formal Conjectures*, file `FormalConjectures/WrittenOnTheWallII/GraphConjecture84.lean`, `https://github.com/google-deepmind/formal-conjectures` (state of 23 October 2026).

[6] A. Ferudun, Three Graffiti.pc conjectures on largest induced trees: proofs of Conjectures 141, 142, and 143, arXiv:2608.01396 (2026).

[7] J. Fox, P.-S. Loh and B. Sudakov, Large induced trees in K_r-free graphs, *J. Combin. Theory Ser. B* 99 (2009), 494–501.

[8] C. Maki, *WOW II #144 in Lean 4* (formal proof including a formalisation of the Bacsó–Tuza Induced Path Lemma), https://github.com/beowulf127/wowii144-lean (2026).

[9] B. D. McKay and A. Piperno, Practical graph isomorphism, II, *J. Symbolic Comput.* 60 (2014), 94–112.

A Source code

The C programs read connected graphs in graph6 format from standard input, for example `geng -cq 10 | ./tlemD`. The $n = 10$ run of `tlemD` was split as `geng -cq 10 r/4` for $r = 0, 1, 2, 3$, and the $n = 10$ runs of `tbt` and `tdiam` as `geng -cq 10 r/2` for $r = 0, 1$. They were compiled with `gcc -O2`. The variant `tlemDr` used for random graphs is `tlemD.c` with an added connectivity filter: the call `parse_g6(line); cnt++;` is replaced by `parse_g6(line); if(!conn((1u<<n)-1)) continue; cnt++;`. The random input was produced by

```
(for p in 12 16 20 25; do genrang -g -P$p/100 15 3000; done; \
 python3 gen_sparse.py 7 16 6000) | ./tlemDr
```

A.1 `tlemD.c`: exhaustive check of Theorems 1.6 and 1.3

```
// Exhaustive check of Lemma D: G connected, k>=1, x with t*=min(dist(x,C_k),k)>=1:
//   (T) tree(G)>=2k+2  or  (R0) exists induced tree T containing x, |T|>=k+t*+1, N_T(x) inside one
    component of G-x.
// Also checks the cut-vertex corollary: G has a cut vertex => tree(G) >= 2 rad(G); and leaf corollary.
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAXN 16
static int n; static unsigned adj[MAXN];
static int popc(unsigned x){return __builtin_popcount(x);}
static void parse_g6(const char*s){ int k=0; n=s[k++]-63; memset(adj,0,sizeof adj); int bits=0; unsigned
    char c=0;
  for(int j=1;j<n;j++) for(int i=0;i<j;i++){ if(bits==0){c=s[k++]-63; bits=6;} bits--; if((c>>bits)&1){adj
    [i]|=1u<<j; adj[j]|=1u<<i;} } }
static int conn(unsigned S){ if(!S) return 0; unsigned seen=S&-S, fr=seen; while(fr){ unsigned nf=0; for(
    unsigned t=fr;t;t&=t-1){int v=__builtin_ctz(t); nf|=adj[v]&S;} nf&=~seen; seen|=nf; fr=nf;} return
    seen==S; }
static unsigned comp(unsigned S, int v){ unsigned seen=1u<<v, fr=seen; while(fr){ unsigned nf=0; for(
    unsigned t=fr;t;t&=t-1){int w=__builtin_ctz(t); nf|=adj[w]&S;} nf&=~seen; seen|=nf; fr=nf;} return
    seen; }
static int edges_in(unsigned S){ int e=0; for(unsigned t=S;t;t&=t-1){int v=__builtin_ctz(t); e+=popc(adj[v
    ]&S);} return e/2; }
static int D[MAXN][MAXN];
static void bfs_all(){ for(int s=0;s<n;s++){ for(int i=0;i<n;i++) D[s][i]=99; D[s][s]=0; unsigned fr=1u<<s,
     seen=fr; int d=0; while(fr){ d++; unsigned nf=0; for(unsigned t=fr;t;t&=t-1){int v=__builtin_ctz(t);
    nf|=adj[v];} nf&=~seen; for(unsigned t=nf;t;t&=t-1) D[s][__builtin_ctz(t)]=d; seen|=nf; fr=nf; } } }
int main(){ char line[256]; long cnt=0, cases=0, failD=0, cutg=0, failcut=0, leafg=0, failleaf=0; static
    unsigned char istree[1<<MAXN];
  while(fgets(line,sizeof line,stdin)){ line[strcspn(line,"\n")]=0; parse_g6(line); cnt++; unsigned full
    =(1u<<n)-1;
   bfs_all(); int ecc[MAXN], rad=99; for(int v=0;v<n;v++){int e=0; for(int w=0;w<n;w++) if(D[v][w]>e) e=D
    [v][w]; ecc[v]=e; if(e<rad) rad=e;}
   int tsz=0; for(unsigned S=1;S<=full;S++){ int kk=popc(S); istree[S]=(kk==1)||(edges_in(S)==kk-1 &&
    conn(S)); if(istree[S]&&kk>tsz) tsz=kk; }
   // rho0[x]
   int rho0[MAXN]; for(int x=0;x<n;x++){ unsigned rest=full&~(1u<<x); unsigned cm[MAXN]; int nc=0;
    unsigned left=rest; while(left){ int v=__builtin_ctz(left); unsigned c=comp(rest,v); cm[nc++]=c; left
    &=~c; }
     int best=1; for(unsigned S=1;S<=full;S++){ if(!((S>>x)&1)||!istree[S]) continue; int p=popc(S); if(p
    <=best) continue; unsigned nb=adj[x]&S; int ok=(nb==0); for(int i=0;i<nc&&!ok;i++) if((nb&~cm[i])==0)
    ok=1; if(ok) best=p; } rho0[x]=best; }
```

```
    for(int k=1;k<n;k++) for(int x=0;x<n;x++){ int dk=99; for(int y=0;y<n;y++) if(ecc[y]<=k && D[x][y]<dk)
      dk=D[x][y]; int t=dk<k?dk:k; if(t<1) continue; cases++;
      if(tsz>=2*k+2) continue; if(rho0[x]>=k+t+1) continue; failD++; if(failD<=10) printf("FAIL-D %s k=%d
     t=%d x=%d\n",line,k,t,x); }
    int hascut=0; for(int v=0;v<n&&!hascut;v++) if(n>=3 && !conn(full&~(1u<<v))) hascut=1;
    if(hascut){ cutg++; if(tsz<2*rad){ failcut++; printf("FAIL-CUT %s\n",line);} }
    int hasleaf=0; for(int v=0;v<n;v++) if(popc(adj[v])==1) hasleaf=1;
    if(hasleaf){ leafg++; if(tsz<2*rad){ failleaf++; printf("FAIL-LEAF %s\n",line);} }
  }
  printf("graphs %ld lemmaD-cases %ld failD %ld | cutgraphs %ld failcut %ld | leafgraphs %ld failleaf %ld\
     n",cnt,cases,failD,cutg,failcut,leafg,failleaf); }
```

A.2 `crosscheck.py`: independent re-implementation

```
# Independent (networkx-based) re-implementation of the Lemma D check, for cross-validation on small orders.

import sys, itertools, networkx as nx
def is_induced_tree(G, S):
    H = G.subgraph(S)
    return len(S) >= 1 and nx.is_tree(H)
def check(G):
    V = list(G); n = len(V)
    E = nx.eccentricity(G)
    trees = [frozenset(S) for r in range(1, n+1) for S in itertools.combinations(V, r) if is_induced_tree(
     G, S)]
    tsz = max(len(S) for S in trees)
    bad = 0; cases = 0
    for x in V:
        Gx = G.copy(); Gx.remove_node(x)
        comps = [set(c) for c in nx.connected_components(Gx)]
        rho = max(len(S) for S in trees if x in S and (not set(G[x]) & S or any(set(G[x]) & S <= c for c in
      comps)))
        D = nx.single_source_shortest_path_length(G, x)
        for k in range(1, n):
            C = [z for z in V if E[z] <= k]
            t = min([D[z] for z in C], default=10**9); t = min(t, k)
            if t < 1: continue
            cases += 1
            if not (tsz >= 2*k+2 or rho >= k+t+1): bad += 1
    rad = min(E.values())
    cut = n >= 3 and len(list(nx.articulation_points(G))) > 0
    return cases, bad, (cut and tsz < 2*rad)
tot = [0,0,0,0]
for line in sys.stdin:
    G = nx.from_graph6_bytes(line.strip().encode())
    c, b, cf = check(G)
    tot[0] += 1; tot[1] += c; tot[2] += b; tot[3] += cf
print("graphs %d cases %d failD %d failcut %d" % tuple(tot))
```

A.3 `sublemmas.py`: randomized checks of the auxiliary steps

Run as `python3 sublemmas.py 1` and `python3 sublemmas.py 2`.

```
# Randomized sanity checks of the auxiliary facts used in the proof of Lemma D.
import random, networkx as nx, itertools, sys
random.seed(int(sys.argv[1]) if len(sys.argv)>1 else 1)

def ecc_all(G):
    return nx.eccentricity(G)

def Ck(G, k, E=None):
    E = E or ecc_all(G)
    return {v for v in G if E[v] <= k}

def far(G, x, k, t):
    D = nx.single_source_shortest_path_length(G, x)
    return all(D[z] >= t for z in Ck(G, k))

def add_pendant(G, v, m, tag):
```

```
    H = G.copy(); prev = v
    for h in range(1, m+1):
        H.add_edge(prev, (tag, h)); prev = (tag, h)
    return H

def rand_conn(n, p):
    while True:
        G = nx.gnp_random_graph(n, p)
        if nx.is_connected(G): return G

bad = {'equiv':0,'caseA':0,'P2':0,'P3d':0,'P3e':0,'Gu':0}
cnt = {k:0 for k in bad}
for it in range(4000):
    n = random.randint(4, 22); p = random.choice([0.08,0.12,0.18,0.25,0.35])
    G = rand_conn(n, p)
    E = ecc_all(G); rad = min(E.values())
    x = random.choice(list(G))
    # 1. Equivalence lemma
    for k in range(1, max(E.values())+2):
        for t in range(1, k+1):
            m = k - t + 1
            Hp = add_pendant(G, x, m, 'P')
            lhs = min(ecc_all(Hp).values()) >= k+1
            rhs = far(G, x, k, t)
            cnt['equiv'] += 1
            if lhs != rhs: bad['equiv'] += 1
    # 2. Case A: s ~ x non-cut, x far in G (t<=k) => x far in G-s
    for k in range(1, max(E.values())+1):
        Dx = nx.single_source_shortest_path_length(G, x)
        C = Ck(G, k, E); tstar = min([Dx[z] for z in C], default=10**9); tstar = min(tstar, k)
        for t in range(1, tstar+1):
            for s in G[x]:
                Gs = G.copy(); Gs.remove_node(s)
                if not nx.is_connected(Gs): continue
                cnt['caseA'] += 1
                if not far(Gs, x, k, t): bad['caseA'] += 1
    # 3. P2: x cut vertex, far by t; e' <= k-1 => far by tau in G1
    if x in set(nx.articulation_points(G)):
        Gx = G.copy(); Gx.remove_node(x); comps = list(nx.connected_components(Gx))
        Dx = nx.single_source_shortest_path_length(G, x)
        depth = [max(Dx[v] for v in c) for c in comps]
        i1 = max(range(len(comps)), key=lambda i: depth[i]); e1 = depth[i1]
        ep = max(depth[i] for i in range(len(comps)) if i != i1)
        G1 = G.subgraph(set(comps[i1]) | {x}).copy()
        for k in range(1, e1):
            if e1 + ep + 1 >= 2*k+2: continue
            C = Ck(G, k, E); tstar = min([Dx[z] for z in C], default=10**9); tstar=min(tstar,k)
            for t in range(1, tstar+1):
                tau = min(t, k+1-ep)
                cnt['P2'] += 1
                if not (1 <= tau <= k and far(G1, x, k, tau)): bad['P2'] += 1
    # 4/5. P3: s neighbor of x, s cut vertex; U, u farthest; G-u connected; if x not far in G-u => mu >= k-t+1;
    #       G~ = G[X+s+q]: x far in G~ provided e_X >= k+1 and mu <= k-1 ... (we test the implication used)
    Dx = nx.single_source_shortest_path_length(G, x)
    for s in G[x]:
        Gs = G.copy(); Gs.remove_node(s)
        if nx.is_connected(Gs): continue
        X = nx.node_connected_component(Gs, x); U = set(G) - X - {s}
        Ds = nx.single_source_shortest_path_length(G, s)
        mu = max(Ds[v] for v in U); u = max(U, key=lambda v: Ds[v])
        Gu = G.copy(); Gu.remove_node(u)
        cnt['Gu'] += 1
        if not nx.is_connected(Gu): bad['Gu'] += 1; continue
        for k in range(1, max(E.values())+1):
            C = Ck(G, k, E); tstar = min([Dx[z] for z in C], default=10**9); tstar=min(tstar,k)
            for t in range(2, tstar+1):
                if not far(Gu, x, k, t):
                    cnt['P3d'] += 1
                    if mu < k-t+1: bad['P3d'] += 1
                # P3e: if mu <= k-1 (so e_X >= k+1 automatically), x far in G~
                if mu <= k-1:
                    path = nx.shortest_path(G, s, u)
```

```
                    Gt = G.subgraph(X | {s} | set(path)).copy()
                    cnt['P3e'] += 1
                    if not far(Gt, x, k, t): bad['P3e'] += 1
print('checked', cnt)
print('violations', bad)
```

A.4 `gen_sparse.py`: random sparse connected graphs

```
import random, sys, networkx as nx
random.seed(int(sys.argv[1])); N=int(sys.argv[2]); cnt=int(sys.argv[3])
out=[]
for _ in range(cnt):
    n=random.randint(10,N)
    T=nx.random_labeled_tree(n) if hasattr(nx,'random_labeled_tree') else nx.random_tree(n)
    G=nx.Graph(T)
    extra=random.choice([0,1,2,3,4,5,6,8,10])
    nodes=list(G)
    for _ in range(extra):
        a,b=random.sample(nodes,2); G.add_edge(a,b)
    out.append(nx.to_graph6_bytes(G,header=False).decode().strip())
print("\n".join(out))
```

A.5 `lemd3.c`: independent checker used for $n = 11$

```
/* Exhaustive check of Lemma D and Theorem A on graphs read in graph6 from stdin.
   Lemma D: G connected, k>=1, 1<=t<=k, x t-far wrt k  =>  tree(G)>=2k+2  or
            exists induced tree T containing x, |T|>=k+t+1, N_T(x) inside one component of G-x.
   Theorem A: G connected with a cut vertex => tree(G) >= 2 rad(G).
   Also records tightness statistics. */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAXN 24
static int n;
static unsigned adj[MAXN];
static int dist[MAXN][MAXN], ecc[MAXN];
static unsigned char *istree;
static long long nskipped=0, ngraphs=0, ninst=0, nfailD=0, nfailA=0, ncut=0, tightA=0, tightD=0;

static int parse_g6(const char *s){
  const unsigned char *p=(const unsigned char*)s;
  if(p[0]=='>' ) { while(*p && *p!='<') p++; p+=2; }
  int nn=p[0]-63; p++;
  memset(adj,0,sizeof adj);
  int x=0,bits=0;
  for(int j=1;j<nn;j++) for(int i=0;i<j;i++){
    if(bits==0){ x=*p++-63; bits=6; }
    bits--;
    if((x>>bits)&1){ adj[i]|=1u<<j; adj[j]|=1u<<i; }
  }
  return nn;
}
static void bfs_all(void){
  for(int s=0;s<n;s++){
    for(int v=0;v<n;v++) dist[s][v]=-1;
    unsigned seen=1u<<s, fr=1u<<s; int d=0; dist[s][s]=0;
    while(fr){ unsigned nf=0; for(int v=0;v<n;v++) if(fr>>v&1) nf|=adj[v]; nf&=~seen; d++;
      for(int v=0;v<n;v++) if(nf>>v&1) dist[s][v]=d; seen|=nf; fr=nf; }
    ecc[s]=0; for(int v=0;v<n;v++){ if(dist[s][v]<0){ecc[s]=1000;} else if(dist[s][v]>ecc[s]) ecc[s]=dist[
     s][v]; }
  }
}
static unsigned reach(unsigned start, unsigned allowed){ /* component of start within allowed */
  unsigned seen=start, fr=start;
  while(fr){ unsigned nf=0; for(int v=0;v<n;v++) if(fr>>v&1) nf|=adj[v]; nf&=allowed&~seen; seen|=nf; fr=
     nf; }
  return seen;
}
```

```
int main(int argc,char**argv){
  char line[512];
  int verbose = argc>1 ? atoi(argv[1]) : 0;
  istree = malloc(1u<<MAXN);
  while(fgets(line,sizeof line,stdin)){
    n=parse_g6(line); ngraphs++;
    { unsigned allm=(1u<<n)-1; int d2=1; for(int u=0;u<n && d2;u++){ unsigned r=adj[u]|(1u<<u); unsigned T
     =adj[u]; while(T){int v=__builtin_ctz(T); T&=T-1; r|=adj[v];} if(r!=allm) d2=0; } if(d2 && !(verbose
     &2)) continue; }
    bfs_all();
    int diam=0, rad=1000; for(int v=0;v<n;v++){ if(ecc[v]>diam) diam=ecc[v]; if(ecc[v]<rad) rad=ecc[v]; }
    if(diam>=1000) continue; /* disconnected */
    unsigned all=(n==32)?0xffffffffu:((1u<<n)-1);
    /* cut vertex? */
    int hascut=0;
    for(int v=0;v<n && !hascut;v++){ unsigned rest=all&~(1u<<v); if(!rest) continue; int s=__builtin_ctz(
     rest);
      if(reach(1u<<s,rest)!=rest) hascut=1; }
    if(diam<=2 && !(verbose&2)) { if(hascut) ncut++; continue; } /* see note: diam<=2 => only k=1,t=1 (
     trivial) and cut=>rad=1 */
    /* quick filter: is any Lemma D instance or the Theorem A check non-trivial? */
    { int need = (hascut && diam+1<2*rad);
      for(int k=1;k<diam && !need;k++){ if(diam>=2*k+1) continue;
        unsigned Ck=0; for(int z=0;z<n;z++) if(ecc[z]<=k) Ck|=1u<<z;
        for(int x=0;x<n && !need;x++){ if(ecc[x]<=k) continue;
          int f=1000; for(int z=0;z<n;z++) if(Ck>>z&1) if(dist[x][z]<f) f=dist[x][z];
          int tmax=f<k?f:k; if(tmax<1) continue;
          if(ecc[x] < k+tmax) need=1; } }
      if(!need){ if(hascut) ncut++; nskipped++; continue; } }
    /* induced trees DP */
    unsigned N=1u<<n; int maxtree=0; int maxR[MAXN]; for(int v=0;v<n;v++) maxR[v]=0;
    /* components of G-x */
    unsigned comp[MAXN][MAXN]; int ncomp[MAXN];
    for(int x=0;x<n;x++){ unsigned rest=all&~(1u<<x); ncomp[x]=0;
      while(rest){ unsigned c=reach(rest&(-rest),all&~(1u<<x)); comp[x][ncomp[x]++]=c; rest&=~c; } }
    istree[0]=0;
    for(unsigned S=1;S<N;S++){
      int sz=__builtin_popcount(S);
      if(sz==1){ istree[S]=1; }
      else { int leaf=-1; unsigned T=S; while(T){ int v=__builtin_ctz(T); T&=T-1; if(__builtin_popcount(
     adj[v]&S)==1){leaf=v;break;} }
        istree[S]= (leaf>=0) ? istree[S^(1u<<leaf)] : 0; }
      if(!istree[S]) continue;
      if(sz>maxtree) maxtree=sz;
      unsigned T=S; while(T){ int x=__builtin_ctz(T); T&=T-1; if(sz<=maxR[x]) continue;
        unsigned nb=adj[x]&S; int ok=0; if(!nb) ok=1; else for(int c=0;c<ncomp[x];c++) if((nb&~comp[x][c])
     ==0){ok=1;break;}
        if(ok) maxR[x]=sz; }
    }
    if(hascut){ ncut++; if(maxtree<2*rad){ nfailA++; printf("FAIL_A %s",line);} else if(maxtree==2*rad)
     tightA++; }
    for(int k=1;k<diam;k++){
      unsigned Ck=0; for(int z=0;z<n;z++) if(ecc[z]<=k) Ck|=1u<<z;
      for(int x=0;x<n;x++){ if(ecc[x]<=k) continue;
        int f=1000; for(int z=0;z<n;z++) if(Ck>>z&1) if(dist[x][z]<f) f=dist[x][z];
        int tmax = f<k? f : k; if(tmax<1) continue;
        for(int t=1;t<=tmax;t++){ ninst++;
          int T_ok = maxtree>=2*k+2, R_ok = maxR[x]>=k+t+1;
          if(!T_ok && !R_ok){ nfailD++; printf("FAIL_D k=%d t=%d x=%d %s",k,t,x,line); }
          else if(!T_ok && maxR[x]==k+t+1 && t>=2) { tightD++; if(verbose&1) printf("TIGHT_D k=%d t=%d x=%
     d %s",k,t,x,line);} }
      }
    }
  }
  fprintf(stderr,"skippedTrivial=%lld graphs=%lld cutgraphs=%lld instances(D)=%lld failD=%lld failA=%lld
     tightA=%lld tightD(t>=2)=%lld\n",
          nskipped,ngraphs,ncut,ninst,nfailD,nfailA,tightA,tightD);
  return 0;
}
```

A.6 gen_sparse2.py: larger sparse graphs

Run as `python3 gen_sparse2.py 11 17 20 600 | ./tlemDr22`.

```
import random, sys, networkx as nx
random.seed(int(sys.argv[1])); lo=int(sys.argv[2]); hi=int(sys.argv[3]); cnt=int(sys.argv[4])
for _ in range(cnt):
    n=random.randint(lo,hi)
    G=nx.Graph(nx.random_labeled_tree(n) if hasattr(nx,'random_labeled_tree') else nx.random_tree(n))
    for _ in range(random.choice([1,2,3,4,5,6,8])):
        a,b=random.sample(list(G),2); G.add_edge(a,b)
    print(nx.to_graph6_bytes(G,header=False).decode().strip())
```

A.7 tbt.c: evidence for Conjecture 7.1

```
// Test BT-tree: for connected H, k>=1, 1<=t<=k, x with dist(x, kcenter)>=t:
//  (R) x starts induced path with >= k+t edges, or (T) H has induced tree with >= 2k+2 vertices.
// Also record cases where only plain BT unrooted (path >= 2k edges) holds.
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAXN 16
static int n; static unsigned adj[MAXN];
static int popc(unsigned x){return __builtin_popcount(x);}
static void parse_g6(const char*s){ int k=0; n=s[k++]-63; memset(adj,0,sizeof adj); int bits=0; unsigned
    char c=0;
  for(int j=1;j<n;j++) for(int i=0;i<j;i++){ if(bits==0){c=s[k++]-63; bits=6;} bits--; if((c>>bits)&1){adj
    [i]|=1u<<j; adj[j]|=1u<<i;} } }
static int dist[MAXN][MAXN];
static void bfs_all(){ for(int s=0;s<n;s++){ for(int i=0;i<n;i++) dist[s][i]=-1; dist[s][s]=0; unsigned fr
    =1u<<s, seen=fr; int d=0; while(fr){ d++; unsigned nf=0; for(unsigned t=fr;t;t&=t-1){int v=
    __builtin_ctz(t); nf|=adj[v];} nf&=~seen; for(unsigned t=nf;t;t&=t-1) dist[s][__builtin_ctz(t)]=d;
    seen|=nf; fr=nf; } } }
static int conn(unsigned S){ if(!S) return 0; unsigned seen=S&-S, fr=seen; while(fr){ unsigned nf=0; for(
    unsigned t=fr;t;t&=t-1){int v=__builtin_ctz(t); nf|=adj[v]&S;} nf&=~seen; seen|=nf; fr=nf;} return
    seen==S; }
static int edges_in(unsigned S){ int e=0; for(unsigned t=S;t;t&=t-1){int v=__builtin_ctz(t); e+=popc(adj[v
    ]&S);} return e/2; }
static int best; static void lip(int v, unsigned S, int len){ if(len>best) best=len; unsigned cand=adj[v]&~
    S; for(unsigned t=cand;t;t&=t-1){ int x=__builtin_ctz(t); if((adj[x]&S)==(1u<<v)) lip(x,S|(1u<<x),len
    +1); } }
int main(){ char line[256]; long cnt=0, cases=0, fail=0;
  while(fgets(line,sizeof line,stdin)){ line[strcspn(line,"\n")]=0; parse_g6(line); cnt++; unsigned full
    =(1u<<n)-1;
    bfs_all(); int ecc[MAXN]; for(int v=0;v<n;v++){int e=0; for(int w=0;w<n;w++) if(dist[v][w]>e) e=dist[v
    ][w]; ecc[v]=e;}
    int tsz=0; for(unsigned S=1;S<=full;S++){ int k=popc(S); if(k<=tsz) continue; if(edges_in(S)==k-1 &&
    conn(S)) tsz=k; }
    int rootlen[MAXN]; for(int x=0;x<n;x++){ best=0; lip(x,1u<<x,1); rootlen[x]=best-1; } // edges
    for(int k=1;k<n;k++){ for(int x=0;x<n;x++){ int dk=99; for(int y=0;y<n;y++) if(ecc[y]<=k && dist[x][y
    ]<dk) dk=dist[x][y];
        int tmax = dk<k?dk:k; for(int t=1;t<=tmax;t++){ cases++; if(rootlen[x]>=k+t) continue; if(tsz>=2*k
    +2) continue; fail++; if(fail<=5) printf("FAIL %s k=%d t=%d x=%d rootlen=%d tree=%d\n",line,k,t,x,
    rootlen[x],tsz); } } }
  }
  printf("graphs %ld cases %ld fail %ld\n",cnt,cases,fail); }
```

A.8 tdiam.c: evidence for Conjecture 7.2

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAXN 16
static int n; static unsigned adj[MAXN];
static int popc(unsigned x){return __builtin_popcount(x);}
static void parse_g6(const char*s){ int k=0; n=s[k++]-63; memset(adj,0,sizeof adj); int bits=0; unsigned
    char c=0;
  for(int j=1;j<n;j++) for(int i=0;i<j;i++){ if(bits==0){c=s[k++]-63; bits=6;} bits--; if((c>>bits)&1){adj
    [i]|=1u<<j; adj[j]|=1u<<i;} } }
```

```
static int dist[MAXN][MAXN];
static void bfs_all(){ for(int s=0;s<n;s++){ for(int i=0;i<n;i++) dist[s][i]=-1; dist[s][s]=0; unsigned fr
    =1u<<s, seen=fr; int d=0; while(fr){ d++; unsigned nf=0; for(unsigned t=fr;t;t&=t-1){int v=
    __builtin_ctz(t); nf|=adj[v];} nf&=~seen; for(unsigned t=nf;t;t&=t-1) dist[s][__builtin_ctz(t)]=d;
    seen|=nf; fr=nf; } } }
static int conn(unsigned S){ if(!S) return 0; unsigned seen=S&-S, fr=seen; while(fr){ unsigned nf=0; for(
    unsigned t=fr;t;t&=t-1){int v=__builtin_ctz(t); nf|=adj[v]&S;} nf&=~seen; seen|=nf; fr=nf;} return
    seen==S; }
static int edges_in(unsigned S){ int e=0; for(unsigned t=S;t;t&=t-1){int v=__builtin_ctz(t); e+=popc(adj[v
    ]&S);} return e/2; }
int main(){ char line[256]; long cnt=0, nsc=0, fail=0, failP=0;
  while(fgets(line,sizeof line,stdin)){ line[strcspn(line,"\n")]=0; parse_g6(line); unsigned full=(1u<<n)
    -1; cnt++;
    bfs_all(); int rad=99, diam=0; for(int v=0;v<n;v++){int e=0; for(int w=0;w<n;w++) if(dist[v][w]>e) e=
    dist[v][w]; if(e<rad) rad=e; if(e>diam) diam=e;}
    if(diam==rad) continue; nsc++;
    int tsz=0, psz=0; for(unsigned S=1;S<=full;S++){ int k=popc(S); if(k<=tsz && k<=psz) continue; if(
    edges_in(S)==k-1 && conn(S)){ if(k>tsz) tsz=k; int ok=1; for(unsigned t=S;t;t&=t-1){int v=
    __builtin_ctz(t); if(popc(adj[v]&S)>2){ok=0;break;}} if(ok && k>psz) psz=k; } }
    if(tsz<2*rad){ fail++; if(fail<=5) printf("FAIL %s rad=%d diam=%d tree=%d\n",line,rad,diam,tsz);}
    if(psz<2*rad){ failP++; if(failP<=3) printf("PATHFAIL %s rad=%d diam=%d path=%d tree=%d\n",line,rad,
    diam,psz,tsz);} }
  printf("graphs %ld non-self-centered %ld: tree<2rad %ld ; path<2rad %ld\n",cnt,nsc,fail,failP); }
```